Autovalutazione — variabili, istruzioni, selezione

Lezioni coperte: `L04` — variabili, istruzioni, espressioni, logica booleana · `L05` — selezione

Quando farla. Da solo, **prima** della sessione `A01`. In aula la commentiamo insieme: chi arriva avendola fatta sa già su che cosa chiedere.

Come funziona. Diciotto domande a risposta multipla. Sotto ciascuna trovi la risposta giusta **e il motivo per cui le altre sono sbagliate**: quelle spiegazioni sono la parte che insegna, quindi leggile anche quando hai indovinato.

Non è un voto. Serve a scoprire che cosa non sai prima che te lo chieda qualcun altro. Se sbagli, segna il numero e portalo in aula.

---

1 — Il tipo della divisione

Di che tipo è il risultato di `10 / 2`?

Mostra la risposta

`float`: vale `5.0`, non `5`.

- *`int`, perché la divisione è esatta* — l'esattezza non c'entra: `/` restituisce **sempre** un float, anche quando il resto è zero. Per ottenere un intero serve `//`;
- *dipende dai valori* — no, il tipo di `/` è deciso dall'operatore, non dai numeri su cui lavora.

---

2 — Quoziente e resto

Quanto valgono `7 // 2` e `7 % 2`?

Mostra la risposta

`3` e `1`.

- *`3.5` e `1`* — `//` tronca, non arrotonda e non restituisce decimali fra interi;
- *`3` e `0.5`* — `%` dà il **resto** della divisione intera, non la parte decimale. Sono due cose diverse: il resto di 7 diviso 2 è 1, perché 2 × 3 + 1 = 7.

---

3 — Il + fra stringhe

Che cosa vale `"12" + "3"`?

Mostra la risposta

`"123"`, una stringa.

- *`15`* — sarebbe vero se fossero numeri. Fra stringhe `+` **concatena**;
- *errore* — no, e questo è il problema: un errore ti avviserebbe. Questa riga produce un risultato plausibile e prosegue.

Succede ogni volta che i valori vengono da `input()`, che restituisce sempre stringhe.

---

4 — Che cosa produce un confronto

Che cos'è `3 > 5`?

Mostra la risposta

Un'espressione che vale `False`, di tipo `bool`.

- *una domanda che si può fare solo dentro un `if`* — no: è un'espressione come `3 + 5`, e si può assegnare a una variabile, stampare, combinare con `and` e `or`. L'`if` è solo uno dei posti in cui si usa;
- *una stringa* — no, `False` non ha virgolette: è uno dei due valori del tipo `bool`.

---

5 — L'assegnazione non è l'uguaglianza

Dopo queste due righe, quanto vale `x`?

```
x = 4
x = x + 3
```

Mostra la risposta

7.

- *non ha soluzione, perché `x` non può essere uguale a `x + 3`* — è vero in algebra, non in Python: `=` non afferma un'uguaglianza, **esegue**. Si calcola prima la destra, `4 + 3`, poi si associa il risultato al nome `x`;
- *4, perché la seconda riga è contraddittoria* — nessuna contraddizione: il vecchio valore serve a calcolare il nuovo e poi non esiste più.

---

6 — `b = a`

Che cosa stampa?

```
a = 7
b = a
a = 99
print(b)
```

Mostra la risposta

7.

- *99, perché `b` è legato ad `a`* — `b = a` non crea un legame che dura: dà a `b` lo stesso oggetto che `a` ha **in quel momento**. Quando `a = 99` associa ad `a` un oggetto nuovo, `b` resta dov'era;
- *errore, perché `b` non è stato aggiornato* — non c'è niente da aggiornare: nessuno ha chiesto a `b` di seguire `a`.

---

7 — Nomi di variabile

Quale di questi **non** è un nome valido in Python 3?

`area_1` · `_totale` · `2lati` · `print`

Mostra la risposta

`2lati`: un nome non può cominciare con una cifra.

- *`_totale`* — valido: l'underscore iniziale è permesso ed è anzi una convenzione;
- *`print`* — **valido**, e qui sta il tranello. In Python 3 `print` è una funzione, non una parola riservata: `print = 5` funziona, e da quel momento `print("ciao")` dà `TypeError: 'int' object is not callable`. Il linguaggio te lo lascia fare: non farlo.

---

8 — Precedenza

Quanto vale `2 + 3 * 4 ** 2`?

Mostra la risposta

50. Prima la potenza — `4 ** 2` fa 16 — poi la moltiplicazione, 48, infine la somma.

- *`400`* — si otterrebbe calcolando da sinistra a destra, `(2+3)*4` poi al quadrato. La precedenza non è l'ordine di lettura;
- *`98`* — si otterrebbe con `(2 + 3 * 4) ** 2`.

Quando il dubbio c'è, si scrivono le parentesi: costano due caratteri.

---

9 — Il meno e la potenza

Quanto vale `-2 ** 2`?

Mostra la risposta

−4. La potenza si applica prima del meno: l'espressione è `-(2 ** 2)`.

- *4* — sarebbe `(-2) ** 2`, che è un'altra espressione;
- *errore* — nessun errore: è perfettamente valida, e questo è il punto.

Non è una regola da ricordare, è una ragione per usare le parentesi.

---

10 — Due booleani sommati

Quanto vale `True + True`?

Mostra la risposta

2. In Python `bool` è un tipo numerico: `True` vale 1 e `False` vale 0.

- *errore, non si sommano i booleani* — si sommano;
- *`True`* — sarebbe `True or True`, che è un'altra operazione.

Non è una curiosità: è il motivo per cui, quando arriveremo a pandas, si contano le righe che soddisfano una condizione **sommando** i confronti.

---

11 — Negare una condizione doppia

Qual è la negazione corretta di `temperatura > 30 and umidita > 70`?

Mostra la risposta

`temperatura <= 30 or umidita <= 70`.

- *`temperatura <= 30 and umidita <= 70`* — è l'errore classico: i confronti sono stati negati bene, uno per uno, ma **`and` non è stato ribaltato in `or`**. Questa versione dice «non fa caldo **e** non è umido», cioè chiede che saltino entrambe le condizioni; «non afoso» vuol dire che ne basta una;
- *`temperatura < 30 or umidita < 70`* — i valori esatti 30 e 70 finirebbero dalla parte sbagliata: la negazione di `> 30` è `<= 30`.

È la legge di **De Morgan**: il `not` entra nelle parentesi e l'operatore in mezzo si ribalta.

---

12 — Quello che entra da tastiera

Dopo `eta = input("quanti anni hai? ")` e l'utente che batte `20`, quanto vale `eta + 1`?

Mostra la risposta

Dà errore: `TypeError: can only concatenate str (not "int") to str`.

- *`21`* — lo sarebbe se `eta` fosse un numero, ma `input()` restituisce **sempre** una stringa, anche quando l'utente batte delle cifre;
- *`"201"`* — sarebbe il risultato di `eta + "1"`, cioè se anche il secondo operando fosse una stringa. Quella versione **non darebbe errore**, ed è il caso più pericoloso.

La correzione è convertire appena si legge: `eta = int(input(...))`.

---

13 — Quale ramo

Che cosa stampa, con `t = 20`?

```
if t > 30:
    print("molto caldo")
elif t > 20:
    print("caldo")
elif t > 10:
    print("mite")
else:
    print("freddo")
```

Mostra la risposta

`mite`.

- *`caldo`* — richiederebbe `t > 20`, e `20 > 20` è **falso**. Il maggiore stretto esclude l'uguaglianza;
- *niente, perché nessuna condizione è vera* — `t > 10` è vera, e la catena si ferma lì.

I valori esatti ai confini sono dove si annidano gli errori, e provando a caso non li si incontra. Con `>=` al posto di `>`, 20 finirebbe in «caldo».

---

14 — `elif` o due `if`

Con `n = 15`, quante righe stampa ciascuna versione?

```
# A
if n % 3 == 0:
    print("tre")
if n % 5 == 0:
```

```
    print("cinque")

# B
if n % 3 == 0:
    print("tre")
elif n % 5 == 0:
    print("cinque")
```

Mostra la risposta

A stampa due righe, B una sola.

- *entrambe due* — `elif` significa «**altrimenti**, se»: viene esaminato solo quando le condizioni precedenti sono false, e per 15 la prima è vera;
- *entrambe una* — due `if` separati sono due domande indipendenti, e possono essere vere tutte e due.

Non è una scelta di stile. «In quale fascia cade?» ha una risposta sola e vuole `elif`; «quali proprietà ha?» può averne più d'una e vuole `if` separati.

---

15 — Un ramo irraggiungibile

Che cosa stampa, con `n = 500`?

```
if n > 0:
    print("positivo")
elif n > 100:
    print("molto grande")
else:
    print("non positivo")
```

Mostra la risposta

`positivo`. Il ramo `molto grande` **non si esegue mai**, per nessun valore di `n`.

- *`molto grande`* — richiederebbe che la catena arrivasse al secondo ramo, ma ogni numero maggiore di 100 è anche maggiore di 0, quindi si ferma al primo;
- *tutte e due le righe* — una catena `elif` esegue **un ramo solo**.

Il programma non dà errori e non avvisa: quelle due righe sono codice morto. In una catena le condizioni vanno dalla **più stretta alla più larga**.

---

16 — L'indentazione

Con `voto = 10`, che cosa stampa?

```python
if voto >= 18:
    print("promosso")
    print("fine")
```

Mostra la risposta

Niente. Entrambe le righe sono dentro il blocco, e il blocco non viene eseguito.

- *`fine`* — lo stamperebbe se quella riga fosse **fuori** dall'`if`, cioè senza i quattro spazi. È una differenza di quattro caratteri che cambia il comportamento del programma;
- *errore* — nessun errore: un `if` che risulta falso semplicemente non fa niente.

In Python l'indentazione non è formattazione, è sintassi: è lei a dire dove comincia e dove finisce un blocco.

---

17 — Semplificare una catena

Si può semplificare questo codice? Come?

```python
if r < 10:
    print("scaglione 1")
elif r >= 10 and r < 20:
    print("scaglione 2")
elif r >= 20 and r < 30:
    print("scaglione 3")
else:
    print("nessuno dei precedenti")
```

Mostra la risposta

Sì: le metà con `>=` si possono togliere.

```python
if r < 10:
    print("scaglione 1")
elif r < 20:
    print("scaglione 2")
elif r < 30:
    print("scaglione 3")
else:
    print("nessuno dei precedenti")
```

- *no, servono per delimitare lo scaglione* — non servono: arrivare al secondo ramo significa già che il primo era falso, cioè che `r >= 10`. La catena **ha già stabilito** il limite inferiore;
- *si può togliere l'`else`* — no: senza, i valori da 30 in su non stamperebbero niente.

Verificato: le due versioni danno la stessa risposta per ogni valore. Scriverne una sola metà non è pigrizia, è dire una cosa una volta sola.

---

18 — Un errore che non c'è più

Questo programma calcola le radici di un'equazione di secondo grado. Eseguendolo con `a=2`, `b=3`, `c=2` — per cui il discriminante vale −7 — che cosa succede?

```
delta = b*b - 4*a*c
rad_delta = delta**0.5
x1 = -(b - rad_delta)/(2*a)
x2 = -(b + rad_delta)/(2*a)
print(x1, x2)
```

Mostra la risposta

Non succede niente di visibile: stampa due numeri complessi, `-0.75+0.66j` e `-0.75-0.66j`.

- *dà errore, perché non si fa la radice di un numero negativo* — è quello che farebbe `math.sqrt(-7)`, che solleva `ValueError`. Ma qui la radice è scritta come `delta**0.5`, e l'elevamento a potenza di un numero negativo in Python 3 restituisce un **complesso** senza protestare;
- *stampa `nan`* — no, `nan` arriva da altre operazioni.

Ed è peggio di un errore. Un'eccezione ferma il programma e la vedi; un complesso prosegue, finisce in una stampa, e magari in una relazione. È il motivo per cui il programma va comunque corretto con un `if` che guarda il segno di `delta` prima di calcolare la radice.

> Se hai visto i video, lì questo esempio dà errore: sono registrati su una versione precedente del linguaggio. Vale quello che leggi qui.

Che cosa portare in aula

Segna i numeri che hai sbagliato. Se sono raggruppati, dicono qualcosa:

se hai sbagliato	rivedi
1, 2, 8, 9	operatori e precedenza — `L04_c`, `L04_d`
3, 12	che `input()` dà stringhe — `L04_c` in coda
5, 6, 7	che cos'è davvero una variabile — `L04_b`, `L04_e`
4, 10, 11	logica booleana — `L04_f`
13, 14, 15, 16, 17	la selezione — `L05_a`
18	niente: era una domanda a tranello, e il tranello è il linguaggio